ISE 305 – Database Systems

Homework 4: Complex SQL Queries

Spring 2026 – Instructor: Ali Çakmak

Yiğit Aydogan 150230208
Emir Mete Dönmez 150220205

May 18, 2026

1 Part I: Database Schema

The NutriQuery database consists of **8 tables** storing USDA FoodData Central and OpenFoodFacts data, brand metadata, nutritional profiles, health/allergen attributes, and machine-learning prediction results.

1.1 Schema Listing

Table	Column	Data Type	Constraint	Description
Brands				
	brand_id	INT	**PK** (IDENTITY)	Surrogate primary key
	brand_name	VARCHAR(255)	NOT NULL	Brand display name
	brand_owner	VARCHAR(255)		Parent company or owner name
FOOD_CATEGORY				
	category_id	INT	**PK** (IDENTITY)	Surrogate primary key
	category_name	VARCHAR(255)	NOT NULL, UNIQUE	Food group name (e.g., Snacks, Beverages)
DATA_TYPE				
	type_id	INT	**PK** (IDENTITY)	Surrogate primary key
	type_name	VARCHAR(100)	NOT NULL, UNIQUE	USDA data source (e.g., SR Legacy, Branded)
Foods				
	fdc_id	INT	**PK**	USDA FoodData Central identifier
	brand_id	INT	**FK** → Brands(brand_id)	Referenced brand (nullable)
	category_id	INT	**FK** → FOOD_CATEGORY(category_id)	Food category (nullable)
	type_id	INT	**FK** → DATA_TYPE(type_id)	Data source type (nullable)
	food_name	VARCHAR(500)	NOT NULL	Descriptive food name
Nutrition_Metrics				
	nutrition_id	INT	**PK** (IDENTITY)	Surrogate primary key
	fdc_id	INT	NOT NULL, **FK** → Foods(fdc_id)	One-to-one with Foods
	calories	FLOAT		Energy (kcal per 100g)
	protein_g	FLOAT		Protein content (g per 100g)
	fat_g	FLOAT		Total fat (g per 100g)
	carbs_g	FLOAT		Carbohydrates (g per 100g)
	sodium_mg	FLOAT		Sodium (mg per 100g)
HEALTH_SCORE				
	score_id	INT	**PK** (IDENTITY)	Surrogate primary key
	fdc_id	INT	NOT NULL, UNIQUE, **FK** → Foods(fdc_id)	One-to-one with Foods
	health_score	FLOAT		Composite health score (0–100)
	nutriscore_grade	VARCHAR(50)		Nutri-Score grade (A–E)
	nova_group	INT		NOVA processing group (1–4)
ALLERGEN_PROFILE				
	allergen_id	INT	**PK** (IDENTITY)	Surrogate primary key
	fdc_id	INT	NOT NULL, UNIQUE, **FK** → Foods(fdc_id)	One-to-one with Foods
	contains_gluten	BIT	DEFAULT 0	Gluten presence flag
	contains_dairy	BIT	DEFAULT 0	Dairy presence flag
ML_Predictions				
	prediction_id	INT	**PK** (IDENTITY)	Surrogate primary key
	fdc_id	INT	NOT NULL, **FK** → Foods(fdc_id)	Predicted food item
	predicted_nutriscore	VARCHAR(50)		ML-predicted Nutri-Score grade
	predicted_nova	INT		ML-predicted NOVA group
	confidence_score	FLOAT		Softmax confidence (0–1)
	prediction_date	DATETIME	DEFAULT GETDATE()	Timestamp of prediction

Table 1: NutriQuery database schema — 8 tables. Primary keys are in **bold**; foreign keys are annotated with their target relation. UNIQUE constraints on `fdc_id` in Nutrition_Metrics, HEALTH_SCORE, and ALLERGEN_PROFILE enforce strict 1:1 relationships.

1.2 Schema Design Summary

- **Lookup Tables:** `FOOD_CATEGORY` and `DATA_TYPE` store category and data-source names independently of `Foods`, avoiding duplication. Category and type names are resolved through foreign keys `category_id` and `type_id`.
- **1:1 Enforcement:** `Nutrition_Metrics`, `HEALTH_SCORE`, and `ALLERGEN_PROFILE` each hold a UNIQUE foreign key on `fdc_id`, guaranteeing that every food has at most one profile in each table.
- **1:N Relationships:** `ML_Predictions` is a 1:N child of `Foods`, allowing multiple predictions per food across retraining runs. `Brands`, `FOOD_CATEGORY`, and `DATA_TYPE` are each 1:N parents of `Foods`.

2 Part II: Complex SQL Queries

We present **6 complex SQL queries** that drive the core analytical features of the NutriQuery web application. Each query joins at least 4 tables and satisfies at least one of the required complexity criteria (JOIN ≥3 tables, OUTER JOIN, GROUP BY, or nested SUBQUERY). All queries were validated against the live MSSQL database.

2.1 Query 1: Full Food Profile Retrieval

SQL Query

```
SELECT
    f.fdc_id,
    f.food_name,
    fc.category_name AS food_category,
    dt.type_name     AS data_type,
    b.brand_name,
    b.brand_owner,
    n.calories,
    n.protein_g,
    n.fat_g,
    n.carbs_g,
    n.sodium_mg,
    hs.health_score,
    hs.nutriscore_grade,
    hs.nova_group,
    ap.contains_gluten,
    ap.contains_dairy
FROM Foods f
LEFT JOIN Brands b             ON f.brand_id    = b.brand_id
LEFT JOIN FOOD_CATEGORY fc     ON f.category_id = fc.category_id
LEFT JOIN DATA_TYPE dt         ON f.type_id     = dt.type_id
LEFT JOIN Nutrition_Metrics n  ON f.fdc_id      = n.fdc_id
LEFT JOIN HEALTH_SCORE hs      ON f.fdc_id      = hs.fdc_id
LEFT JOIN ALLERGEN_PROFILE ap  ON f.fdc_id      = ap.fdc_id
WHERE f.fdc_id = @fdc_id;
```

Query Translation

- **English Meaning:** Retrieves the complete profile of a single food item identified by its USDA `fdc_id`. It assembles brand metadata (via Brands), category and data source labels (via FOOD_CATEGORY and DATA_TYPE lookup tables), full nutrition facts, health scores, and allergen flags into one unified result row. LEFT JOINs ensure that foods with missing brand, category, or health data are still returned rather than silently dropped.
- **Project Feature:** Powers the **"Lookup by FDC ID"** search bar in the Product Registry panel on the NutriQuery home page. The frontend calls `GET /foods/{fdc_id}` and renders the full nutritional breakdown.
- **Criteria Met:**
  1. JOIN ≥3 tables — joins **Foods**, **Brands**, **FOOD_CATEGORY**, **DATA_TYPE**, **Nutrition_Metrics**, **HEALTH_SCORE**, and **ALLERGEN_PROFILE** (7 tables).
  2. OUTER JOIN — uses six LEFT JOINs to preserve food records even when related rows in lookup or profile tables are absent.

2.2 Query 2: Multi-Constraint Range Query

SQL Query

```
SELECT
    f.fdc_id,
    f.food_name,
    fc.category_name AS food_category,
    b.brand_name,
    n.calories,
    n.protein_g,
    n.fat_g,
    n.carbs_g,
    n.sodium_mg,
    hs.health_score,
    hs.nutriscore_grade
FROM Foods f
JOIN FOOD_CATEGORY fc        ON f.category_id = fc.category_id
JOIN Nutrition_Metrics n     ON f.fdc_id      = n.fdc_id
JOIN HEALTH_SCORE hs         ON f.fdc_id      = hs.fdc_id
LEFT JOIN Brands b           ON f.brand_id    = b.brand_id
WHERE hs.health_score >= @min_health_score
  AND n.sodium_mg    <= @max_sodium
  AND n.carbs_g      <= @max_carbs
ORDER BY hs.health_score DESC;
```

Query Translation

- **English Meaning:** Finds foods that simultaneously satisfy three user-defined nutritional thresholds: a minimum health score, a maximum sodium level, and a maximum carbohydrate level. Results are ranked from healthiest to least healthy. INNER JOINs are used for nutrition, health, and category data because a food without these metrics is irrelevant to a range-filtering use case.
- **Project Feature:** Drives the **Range Query** panel in the Nutrition Analytics section. Users adjust sliders for health score, sodium, and carbs; the frontend calls `GET /queries/range` and renders the filtered list in an AG Grid table.
- **Criteria Met:**
  1. JOIN ≥3 tables — joins **Foods**, **FOOD_CATEGORY**, **Nutrition_Metrics**, **HEALTH_SCORE**, and **Brands** (5 tables).
  2. OUTER JOIN — LEFT JOIN on Brands allows foods without a brand assignment to still appear in results.

2.3 Query 3: Category Nutritional Aggregation

SQL Query

```
SELECT
    fc.category_name AS food_category,
    AVG(n.calories)  AS avg_calories,
    AVG(n.protein_g) AS avg_protein,
    AVG(n.fat_g)     AS avg_fat,
    AVG(n.carbs_g)   AS avg_carbs,
    COUNT(f.fdc_id)  AS item_count
FROM Foods f
JOIN FOOD_CATEGORY fc     ON f.category_id = fc.category_id
JOIN Nutrition_Metrics n  ON f.fdc_id      = n.fdc_id
WHERE fc.category_name = @category
GROUP BY fc.category_name;
```

Query Translation

- **English Meaning:** Computes aggregate nutritional statistics — average calories, protein, fat, and carbohydrates — for all food items belonging to a user-selected category. The COUNT function reports the total number of items in that category with complete nutrition data. Renaming a category requires only a single-row update in FOOD_CATEGORY with no cascading changes to Foods.
- **Project Feature:** Populates the **Category Aggregation** dashboard in the Nutrition Analytics section. The user picks a category from a dropdown (populated by `GET /categories/`), and the frontend renders five summary stat cards.
- **Criteria Met:**
  - **1** JOIN ≥3 tables — joins **Foods**, **FOOD_CATEGORY**, and **Nutrition_Metrics** (3 tables).
  - **3** GROUP BY — uses `GROUP BY fc.category_name` with four `AVG()` aggregates and `COUNT()`.

2.4 Query 4: Dietary Restriction Filtering

SQL Query

```
SELECT
    f.fdc_id,
    f.food_name,
    fc.category_name AS food_category,
    b.brand_name,
    n.calories,
    n.protein_g,
    n.fat_g,
    n.carbs_g,
    hs.health_score,
    hs.nutriscore_grade
FROM Foods f
JOIN FOOD_CATEGORY fc       ON f.category_id = fc.category_id
JOIN ALLERGEN_PROFILE ap    ON f.fdc_id      = ap.fdc_id
LEFT JOIN HEALTH_SCORE hs   ON f.fdc_id      = hs.fdc_id
LEFT JOIN Nutrition_Metrics n ON f.fdc_id     = n.fdc_id
LEFT JOIN Brands b          ON f.brand_id    = b.brand_id
WHERE ap.contains_gluten = 0
  AND ap.contains_dairy  = 0
ORDER BY f.food_name;
```

Query Translation

- **English Meaning:** Retrieves all foods that are free from both gluten and dairy, together with their category label, nutrition facts, health score, and brand. The allergen filter uses an INNER JOIN on ALLERGEN_PROFILE to guarantee the food has a recorded allergen status, while LEFT JOINs on HEALTH_SCORE, Nutrition_Metrics, and Brands ensure that foods lacking those profiles still appear — the user is filtering by dietary restriction, not by data completeness.
- **Project Feature:** Powers the **Dietary Filtering** sub-panel in the Nutrition Analytics section. Users toggle "Gluten-Free" and "Dairy-Free" checkboxes; the frontend calls `GET /queries/dietary` and displays matching foods.
- **Criteria Met:**
  1. JOIN ≥3 tables — joins **Foods**, **FOOD_CATEGORY**, **ALLERGEN_PROFILE**, **HEALTH_SCORE**, **Nutrition_Metrics**, and **Brands** (6 tables).
  2. OUTER JOIN — three LEFT JOINs preserve rows that lack health, nutrition, or brand data.

2.5 Query 5: Above-Category-Average Calorie Detection

SQL Query

```
SELECT
    f.fdc_id,
    f.food_name,
    fc.category_name AS food_category,
    n.calories,
    n.protein_g,
    n.fat_g,
    n.carbs_g,
    hs.health_score,
    hs.nutriscore_grade,
    b.brand_name
FROM Foods f
JOIN FOOD_CATEGORY fc        ON f.category_id = fc.category_id
JOIN Nutrition_Metrics n     ON f.fdc_id      = n.fdc_id
LEFT JOIN HEALTH_SCORE hs    ON f.fdc_id      = hs.fdc_id
LEFT JOIN Brands b           ON f.brand_id    = b.brand_id
WHERE n.calories > (
    SELECT AVG(n2.calories)
    FROM Foods f2
    JOIN Nutrition_Metrics n2 ON f2.fdc_id = n2.fdc_id
    WHERE f2.category_id = f.category_id
      AND n2.calories IS NOT NULL
)
ORDER BY fc.category_name, n.calories DESC;
```

Query Translation

- **English Meaning:** Identifies food items whose calorie count exceeds the average calorie count of all foods in the *same* category. For each food, a correlated subquery dynamically computes its category's average calories using the `category_id` foreign key for efficient correlation, and the outer query returns only those above that threshold. This highlights surprisingly energy-dense products within each food group.
- **Project Feature:** Drives an **"Above-Average Calories"** insight panel in the Nutrition Analytics section. Users see which products are unusually high in calories relative to their category peers, aiding diet planning and nutritional awareness.
- **Criteria Met:**
  1. JOIN ≥3 tables — outer query joins **Foods**, **FOOD_CATEGORY**, **Nutrition_Metrics**, **HEALTH_SCORE**, and **Brands** (5 tables); the subquery joins **Foods** and **Nutrition_Metrics**.
  2. OUTER JOIN — two LEFT JOINs in the outer query.
  3. Nested SUBQUERY — the `WHERE` clause contains a correlated subquery that computes the per-category average by referencing the outer

```
        f.category_id.
```

2.6 Query 6: Nutritional Gap Identification

SQL Query

```
SELECT
    f.fdc_id,
    f.food_name,
    fc.category_name AS food_category,
    dt.type_name     AS data_type,
    b.brand_name,
    n.calories,
    n.protein_g,
    n.fat_g,
    n.carbs_g,
    n.sodium_mg,
    hs.health_score,
    hs.nutriscore_grade
FROM Foods f
JOIN Nutrition_Metrics n     ON f.fdc_id      = n.fdc_id
LEFT JOIN FOOD_CATEGORY fc   ON f.category_id = fc.category_id
LEFT JOIN DATA_TYPE dt       ON f.type_id     = dt.type_id
LEFT JOIN HEALTH_SCORE hs    ON f.fdc_id      = hs.fdc_id
LEFT JOIN Brands b           ON f.brand_id    = b.brand_id
WHERE n.calories   IS NULL
   OR n.protein_g IS NULL
   OR n.fat_g     IS NULL
   OR n.carbs_g   IS NULL
   OR n.sodium_mg IS NULL
ORDER BY f.food_name;
```

Query Translation

- **English Meaning:** Surfaces foods whose Nutrition_Metrics record contains at least one NULL value among the five core nutrient fields (calories, protein, fat, carbs, sodium). It identifies data-quality gaps that need attention during the import or enrichment pipeline. Category, data type, health, and brand data are brought in via LEFT JOINs to provide full context without excluding any incomplete records.
- **Project Feature:** Serves the **Gap Identification** panel in the Nutrition Analytics section. The user clicks "Identify Gaps," the frontend calls `GET /queries/gaps`, and the application lists all foods with incomplete nutritional data so the team can target them for enrichment.
- **Criteria Met:**
  1 JOIN ≥3 tables — joins **Foods**, **Nutrition_Metrics**, **FOOD_CATEGORY**, **DATA_TYPE**, **HEALTH_SCORE**, and

Brands (6 tables).

2 OUTER JOIN — four LEFT JOINs preserve context from lookup and profile tables even when those rows are missing.

3 Criteria Coverage Summary

Query	Table Count	(1) JOIN ≥3	(2) OUTER JOIN	(3) GROUP BY	(4) SUBQUERY
Q1: Full Food Profile	7	✓	✓		
Q2: Multi-Constraint Range	5	✓	✓		
Q3: Category Aggregation	3	✓		✓	
Q4: Dietary Filtering	6	✓	✓		
Q5: Above-Average Calories	5	✓	✓		✓
Q6: Gap Identification	6	✓	✓		

Table 2: All six queries join at least 3 tables. Each of the four complexity criteria is covered by at least two queries. Criterion 4 is demonstrated by Q5's correlated subquery that references the outer `f.category_id`. Queries Q1–Q4 and Q6 are currently implemented in the NutriQuery backend (`crud.py`); Q5 is planned for the next release.